



Data Warehouses

Lecture 15

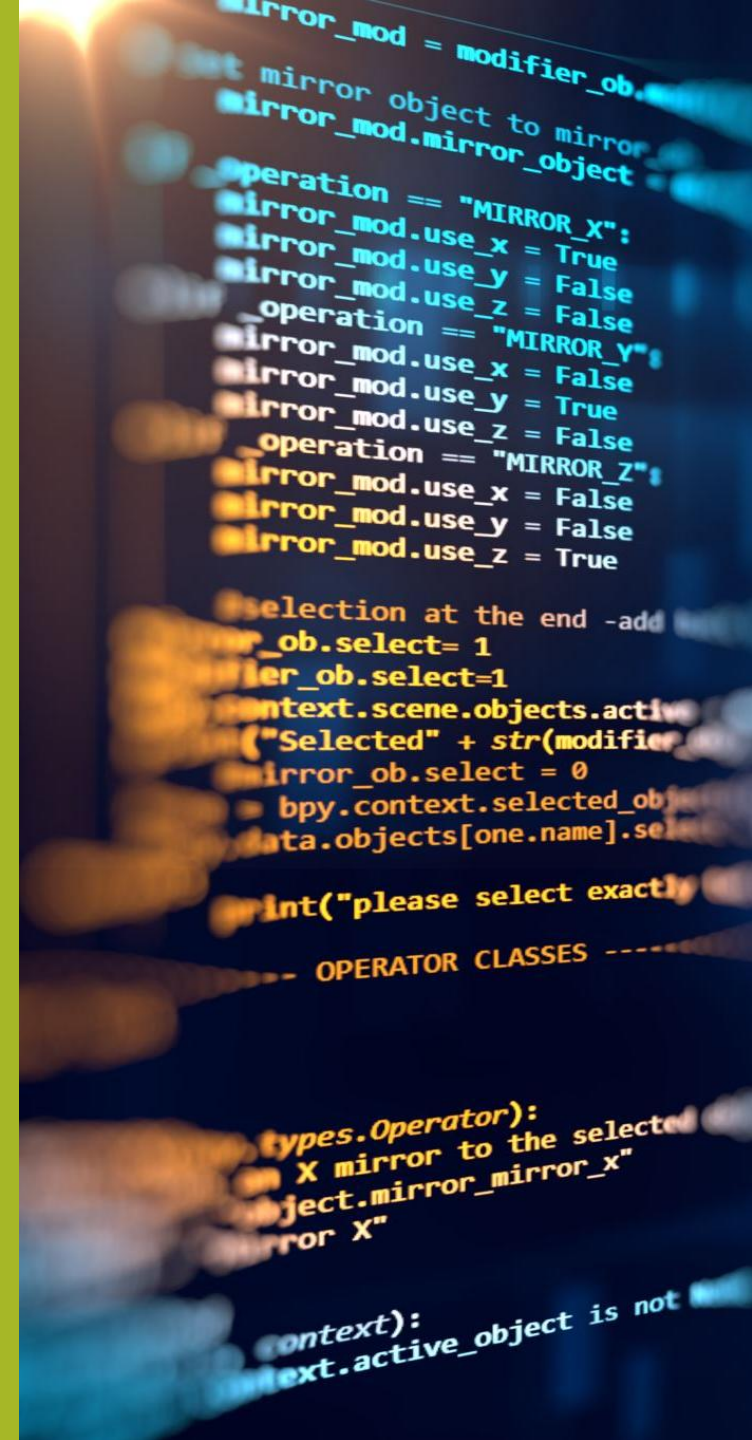
Lecture Goals

Goal:

Synopsis of querying a DW – MDX overview

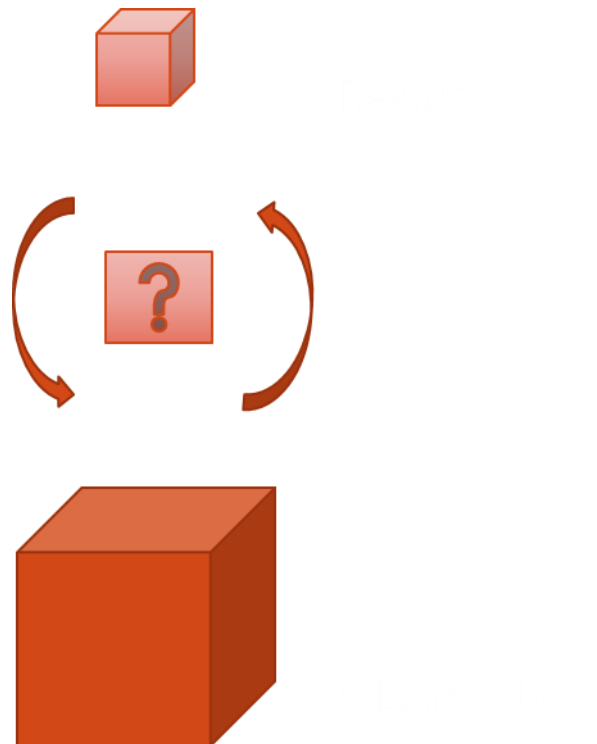
Querying Data Warehouses

Basics of DW querying, Basic
MDX Concepts, Basic MDX
Queries

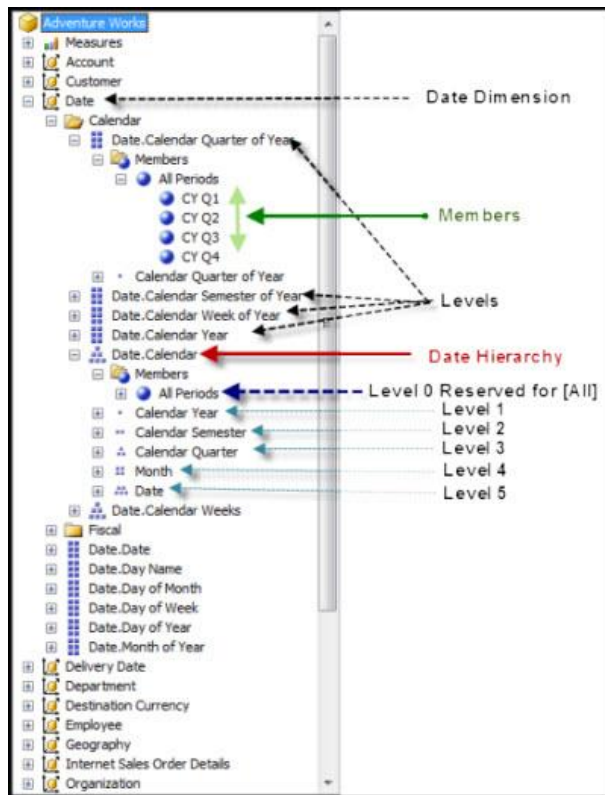


MDX

MDX Query Language is used to retrieve information stored in OLAP Cube



MDX



OLAP Cube

Dimensions

Levels

attributes under Dimension are considered as levels

are also called as Attribute Hierarchy.

Members

Each Level contains one or more members.

Hierarchies

Contains various levels

Default level 0 – represents ALL

MDX

MDX Query

SELECT

*axis specification ON COLUMNS,
axis specification ON ROWS,
etc*

FROM cube_name / subcube

WHERE slicer_specification

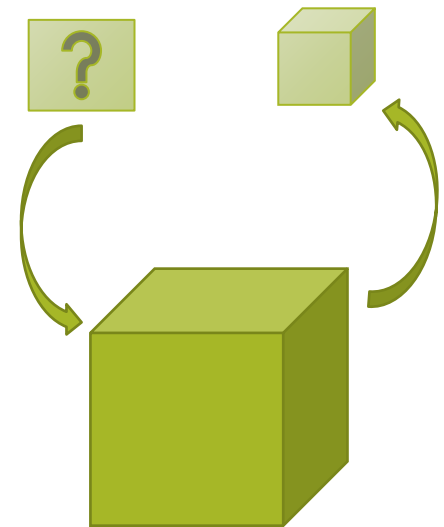
Example

SELECT

*[Time].[1997],[Time].[1998] ON COLUMNS,
[Measures].[Sales],[Measures].[Cost] ON ROWS*

FROM Warehouse

WHERE ([Store].[All Stores].[USA])



MDX

WITH Clause, Named Sets, and Calculated Members

WITH clause provides you with the ability to create calculations that must be formulated within the scope of a specific query

Typical calculations that are created using the WITH clause are named sets and calculated members

You can specify several calculations in one WITH clause

Comma separated

The syntax of the WITH clause is:

[WITH <formula_expression> [, <formula_expression> ...]]

Hierarchy Navigation

Parent

opposite of the Children function.

Parent function

Returns the parent of the specified member.

Returns a null entity if the member has no parent

it is a root member

Member.Parent

Query

[Geography].[Geo].[Canada].Parent

Result

{[Geography].[Geo].[Continent].[America]}



MDX

Dimension

[Customer]

Hierarchy

[Customer].[Geography]

Levels

[Customer].[Geography].[Region]

[Customer].[Geography].[Country]

[Customer].[Geography].[City]



Every hierarchy has an ALL level by default

Hierarchies		
Calendar	Fiscal	ISO Week
▪ Calendar Year	▪ Fiscal Year	▪ ISO Week Number
•• Calendar Month	•• Fiscal Month	•• Date
••• Date	••• Date	<new level>
<new level>	<new level>	

MDX

MDX CONCEPTS

Levels

DIM



All Products

PCategory

PName



Members

DIM



All Products

Drink

Food

D1

D2

D3

F1

F2

MDX

Levels

- ▼ **LS** Levels
 - LA** All Regions Level
 - L** Region
 - L** Country
 - L** City

Members

- ▼ **AD** All Regions
 - ▼ **M** Africa
 - ▼ **M** Egypt
 - M** Cairo
 - ▼ **M** South Africa
 - M** Pretoria
 - ▶ **M** Asia
 - ▶ **M** Europe
 - ▶ **M** North America
 - ▶ **M** Oceania
 - ▶ **M** South America

MDX

OLAP cube

Measure group

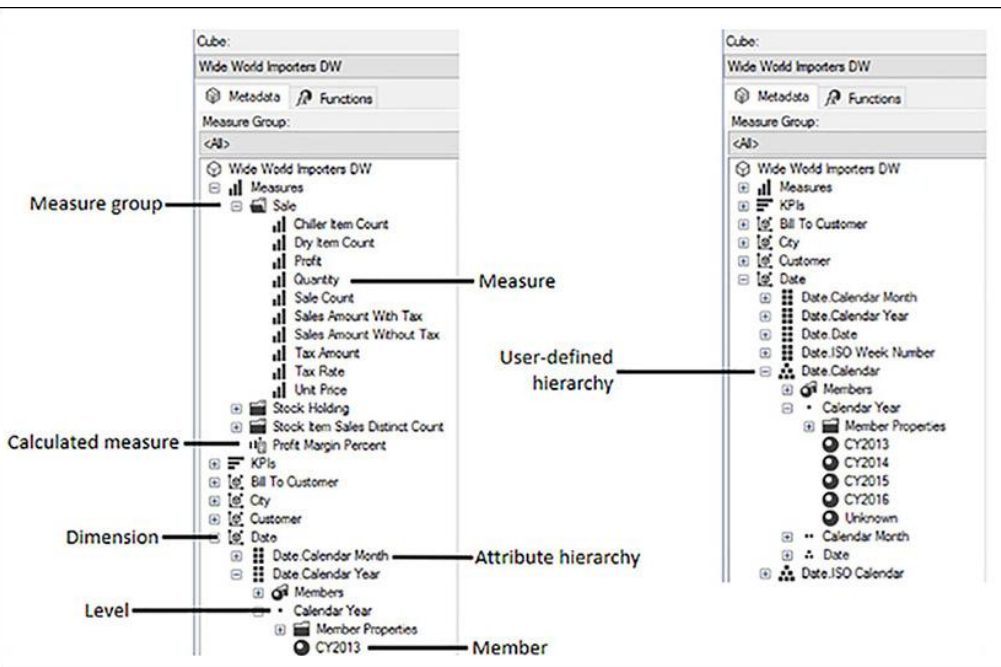
Collection of measures.

Measure

Numeric value, typically aggregated.

Calculated measure

Calculated value, typically aggregated.



MDX

Members and sets

A member

a particular value of an attribute within a dimension

Reference a member

*use the **member's qualified name***

an identifier for a member that uniquely identifies it within a cube

includes both the dimension and hierarchy name as prefixes to the member, uses a period (.) as a delimiter between object names, and encloses each object name in brackets:

[Stock Item].[Color].[Black]

*use its **key column value**, also known as its unique name*

each attribute has a KeyColumn and NameColumn property

syntax for a dimension member's unique name appends an ampersand (&) prior to the KeyColumn value:

[Stock Item].[Stock Item].&[1]

MDX

Member Names / Keys

`[Time].[2nd half].[4th quarter]`

*example referenced the member by its member name,
4th quarter, in the Time dimension.*

`[Time].[2nd half].&[Q4]`

example reference by the member key.

the member key of the 4th quarter member, Q4, is used.

`[Date].[Calendar].[Calendar Quarter].&[2006]&[1]`

the key path (using the key of the member)

The member name can be duplicated or it can be changed in the case of changing dimensions

Referencing the member key ensures proper member identification

MDX

MDX Concepts

access a member in a dimension - path

*with its dimension name, hierarchy name, and level name
using the name path (using the name of the member)*

[Customer].[Geography].[Europe]

is a region (with all European countries as children)

[Customer].[Geography].[Europe].[Poland]

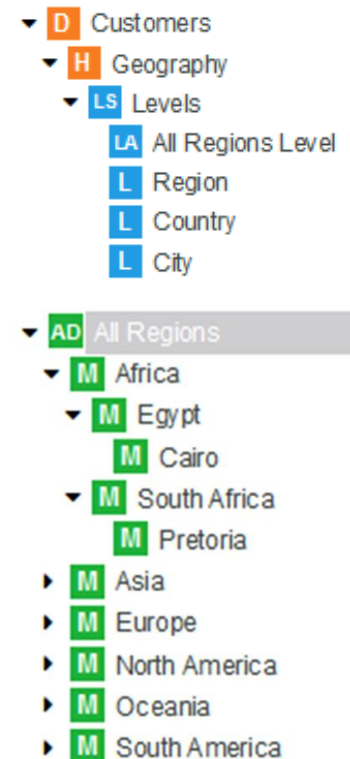
is a country whose parent member is Europe)

[Customer].[Geography].[Europe].[Poland].[Warsaw]

is a city (whose parent member is Poland)

[Customers].[Geography].[Warsaw]

is possible if Warsaw is unique name



MDX

Members and sets

A set

is a collection of members from the same dimension and hierarchy.

Cannot combine attributes from separate hierarchies of the same dimension into a single set.

Used hierarchy can be an attribute hierarchy or a user-defined hierarchy.

a very important concept in MDX

Measures can also be members of a set

Type of set	Set example
Empty set	{}
Single member	[Stock Item].[Color].[Black]
Multiple members	{[Stock Item].[Color].[Black], [Stock Item].[Color].[Blue]}
Range of members	{ [Invoice Date].[Calendar].[CY2016-Jan] : [Invoice Date].[Calendar].[CY2016-Mar] }
Function	[Stock Item].[Color].Members

MDX

Members and sets

Order

Explicitly defining the members in a set by listing the members and enclosing them in brackets

the order in which the members appear in the set is the order in which they appear when you add them to the query.

Using a set function like Members

the OrderBy property for the attribute determines the sort order of those members in the query results.

MDX

To run a MDX query in SSMS against Analysis Services:

Connect to Analysis Service instance

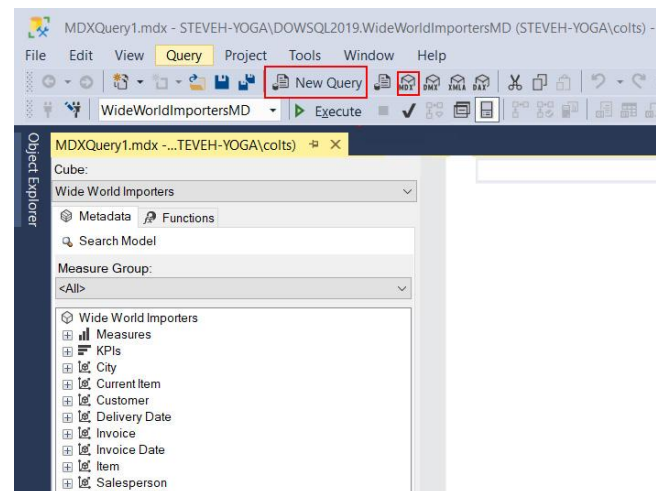
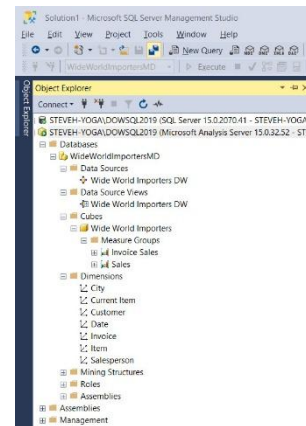
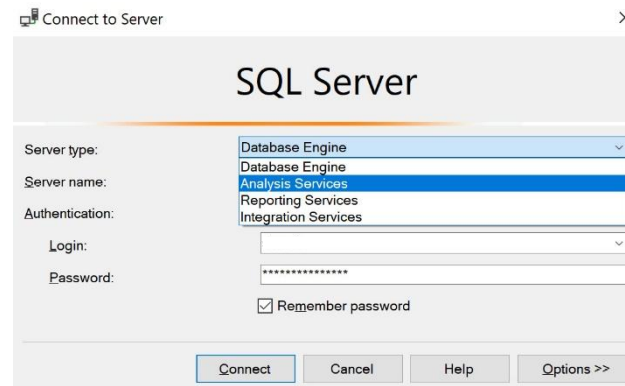
which contains a cube that you want to query

Right-click on the cube and go to New Query | MDX

This will open a new query window.

After typing the query text

you can execute queries by pressing the F5 key or by clicking on the execute button (includes an exclamation mark).



MDX

MDX Query

SELECT

*axis specification ON COLUMNS,
axis specification ON ROWS,
etc*

FROM cube_name / subcube

WHERE slicer_specification

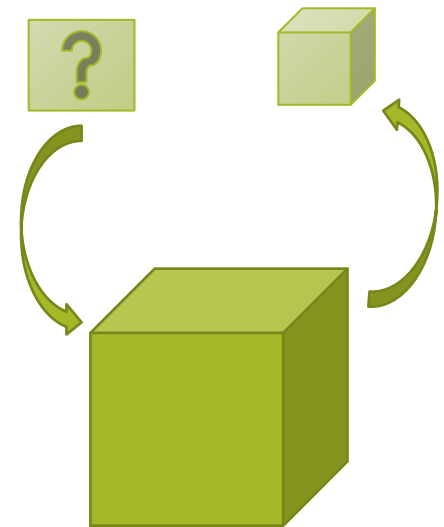
Example

SELECT

*[Time].[1997],[Time].[1998] ON COLUMNS,
[Measures].[Sales],[Measures].[Cost] ON ROWS*

FROM Warehouse

WHERE ([Store].[All Stores].[USA])



MDX

In short:

SELECT:

*axes dimensions, on columns and rows (pages, chapters, and sections)
you cannot skip an axis in a query (consecutive numbering)*

FROM:

Data source cube specification

If joined, data cubes must share dimensions

WHERE:

Slicer - restricts the data area

No axis or the WHERE statement can share any of the same dimensions

Axes

MDX queries can have 0, 1, 2 or up to 129 query axes in the SELECT statement.

Each axis behaves in exactly the same way

unlike SQL where there are significant differences between how the rows and the columns of a query behave.

Naming convention

Axis	Alias
0	Columns
1	Rows
2	Pages
3	Section
4	Chapter

A simple query:

SELECT

[Measures].[Quantity] ON COLUMNS,

[Stock Item].[Color].[Black] ON ROWS

FROM

[Wide World Importers DW];



	Quantity
Black	1,101,101

MDX

An MDX query must contain the following information:

The number of **axes** on which the result is presented.

The set of **members or tuples** to include on each axis of the MDX query.

The name of the **cube** that sets the context of the MDX query.

Additionally can contain:

The set of **members or tuples** to include on the slicer axis.

```
SELECT
  [Time].[1997],[Time].[1998]
ON COLUMNS,
  [Measures].[Sales],[Measures].[Cost]
ON ROWS
FROM
  Warehouse
WHERE
  ([Store].[All Stores].[USA])
```

The diagram illustrates the mapping of MDX query components to their descriptions. Arrows point from the descriptions on the left to the corresponding parts of the query on the right:

- The description "The number of **axes** on which the result is presented." points to the column and row specifications: `[Time].[1997],[Time].[1998]` on columns and `[Measures].[Sales],[Measures].[Cost]` on rows.
- The description "The set of **members or tuples** to include on each axis of the MDX query." points to the same column and row specifications.
- The description "The name of the **cube** that sets the context of the MDX query." points to `Warehouse` in the `FROM` clause.
- The description "Additionally can contain: The set of **members or tuples** to include on the slicer axis." points to the `WHERE` clause: `([Store].[All Stores].[USA])`.

Another simple query:

SELECT

{[Measures].[Sales Amount Without Tax], [Measures].[Profit]} ON COLUMNS,

{[Stock Item].[Color].[Black], [Stock Item].[Color].[Blue]} ON ROWS

FROM

[Wide World Importers DW];

	Sales Amount Without Tax	Profit
Black	\$26,269,280.00	\$14,390,527.00
Blue	\$35,903,882.00	\$17,713,073.00

MDX

MDX CONCEPTS

Tuple

is a collection of members from different dimensions and hierarchies that is enclosed in parentheses

MDX expression that uniquely identifies a cell

[or more generally a section of a cube (slice)]

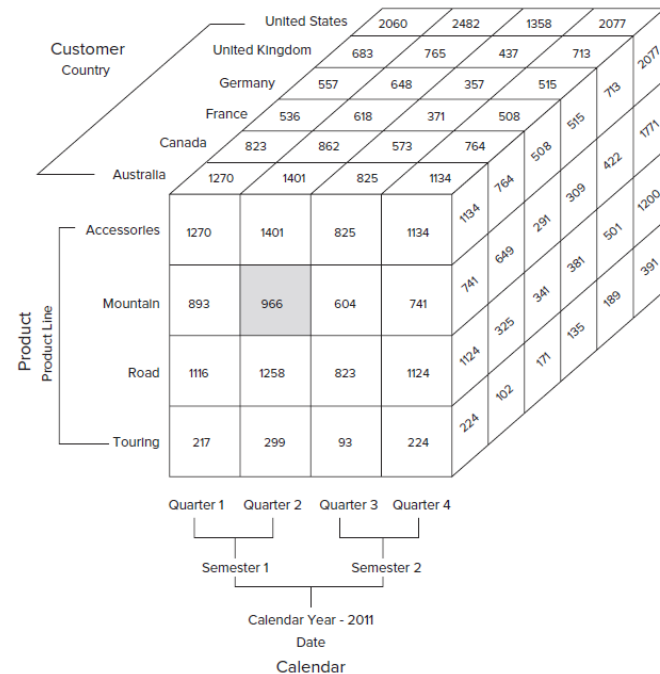
represented by one member from each dimension

separated by a comma

enclosed within parentheses - (...)

tuple does not necessarily need to explicitly contain members from all the dimensions

default values are utilised



MDX

MDX CONCEPTS

Tuple

A tuple represents the address of a location within a multidimensional model

Easiest way to understand a tuple is to look at an Excel sheet

A cell is identified by A1

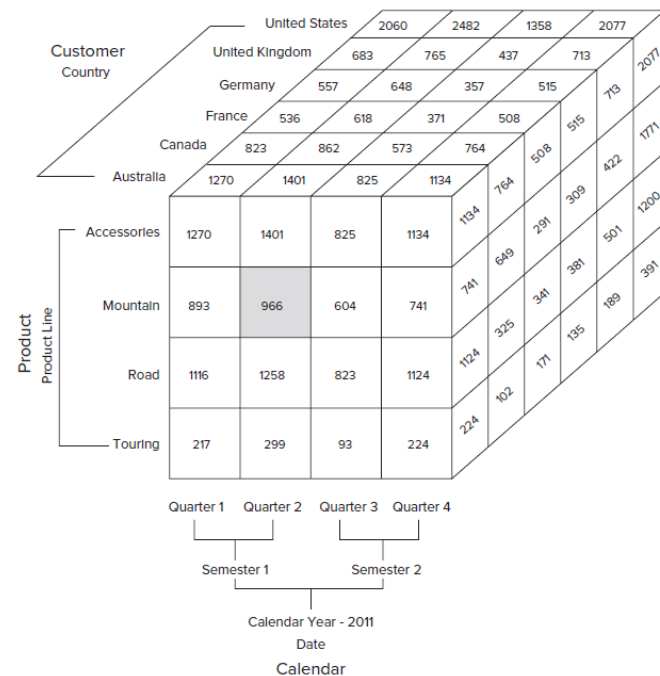
A represents a column coordinate

1 represents a row coordinate

A1 is an example of a tuple

Tuples implicitly include all dimensions in the address

typically the ALL level (default)



MDX

MDX CONCEPTS

Cell (shaded area)

*The Sales amount value in this cell is **966**.*

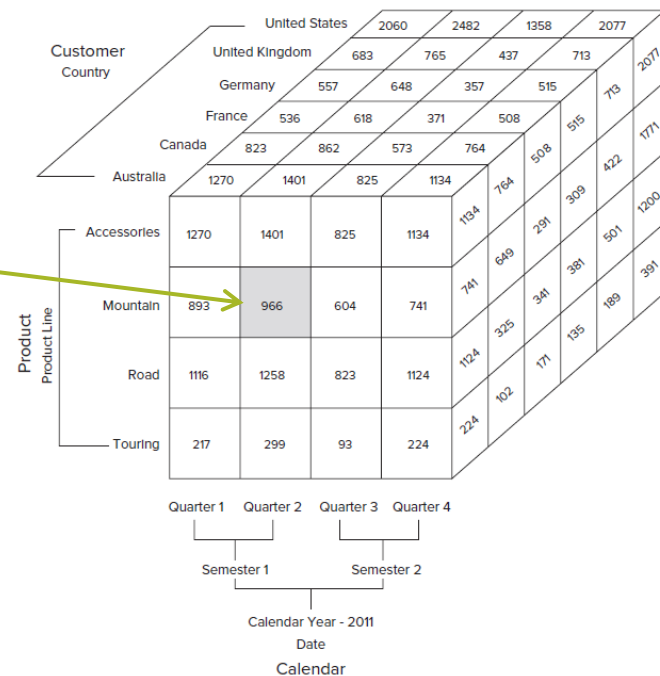
This cell is located at the intersection of

Mountain, Quarter2, and Australia

Tuple

(
 [Product].[Mountain],
 [Date].[Quarter2],
 [Customer].[Australia]
)

the order of the members used to represent a tuple does not matter



To retrieve the Sales Amount for the North territory in the month of January:

```
(  
  [Territory].[Territory].[North],  
  [Date].[Month].[January],  
  [Measures].[Sales Amount]  
)
```

	All Dates	January	February	March	
All Territories					Sales Amount
East					
West					
South					
North					

MDX

MDX CONCEPTS

Tuple

a tuple uniquely identifies a slice

it **cannot** contain more than one member from each dimension

Examples

Cube – Time (1 attr.), Product (2 attr.), Country (1 attr.)

([time].[year].[2011],[product].[product color].[green])

Cube space consisting of all the green products and the year 2011

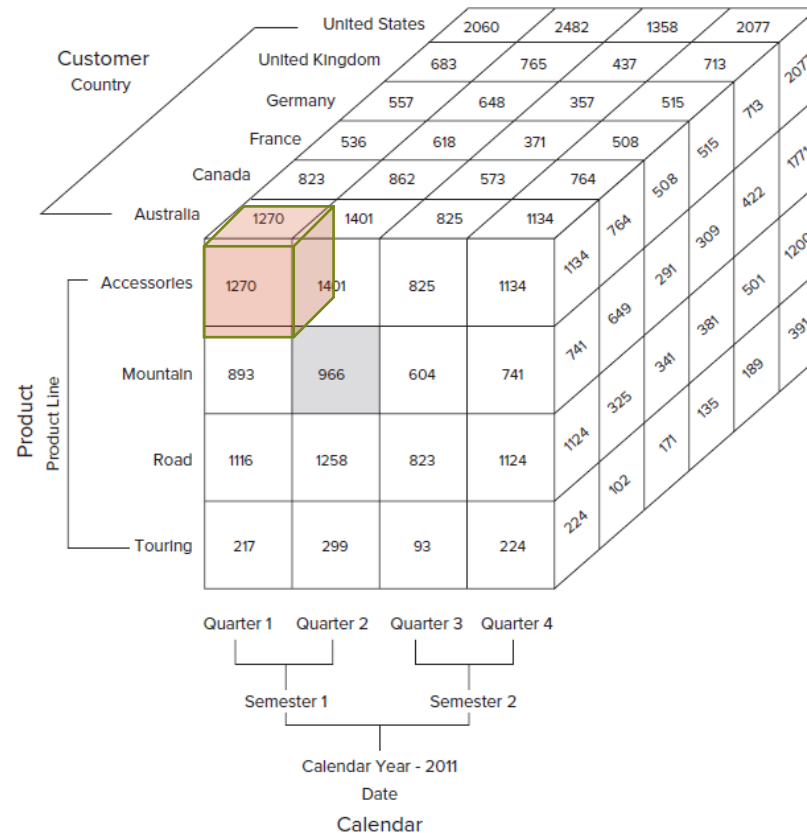
([country].[country].[England])

Cube space consisting of country England

MDX

MDX CONCEPTS

A tuple identifies a single cell or multiple cells using a combination of dimensions.



MDX

MDX CONCEPTS

Tuple

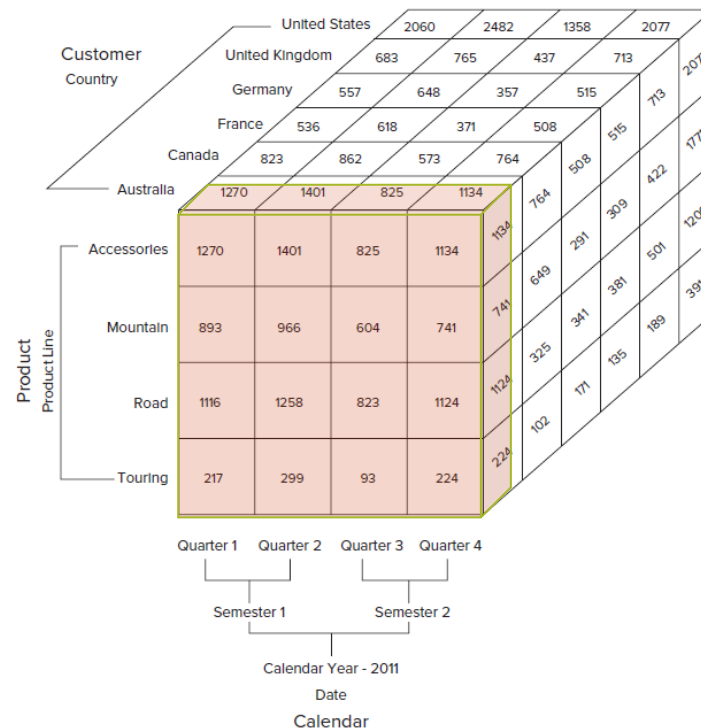
*When you refer to the tuple
([Customer].[Country].[Australia])*

you actually refer to the 16 cells that
correspond to the country Australia

*Therefore when you retrieve the
data held by the cells,*

you are retrieving the measure
values of all the customers in
Australia

It is an aggregate of the cells
(shaded area)

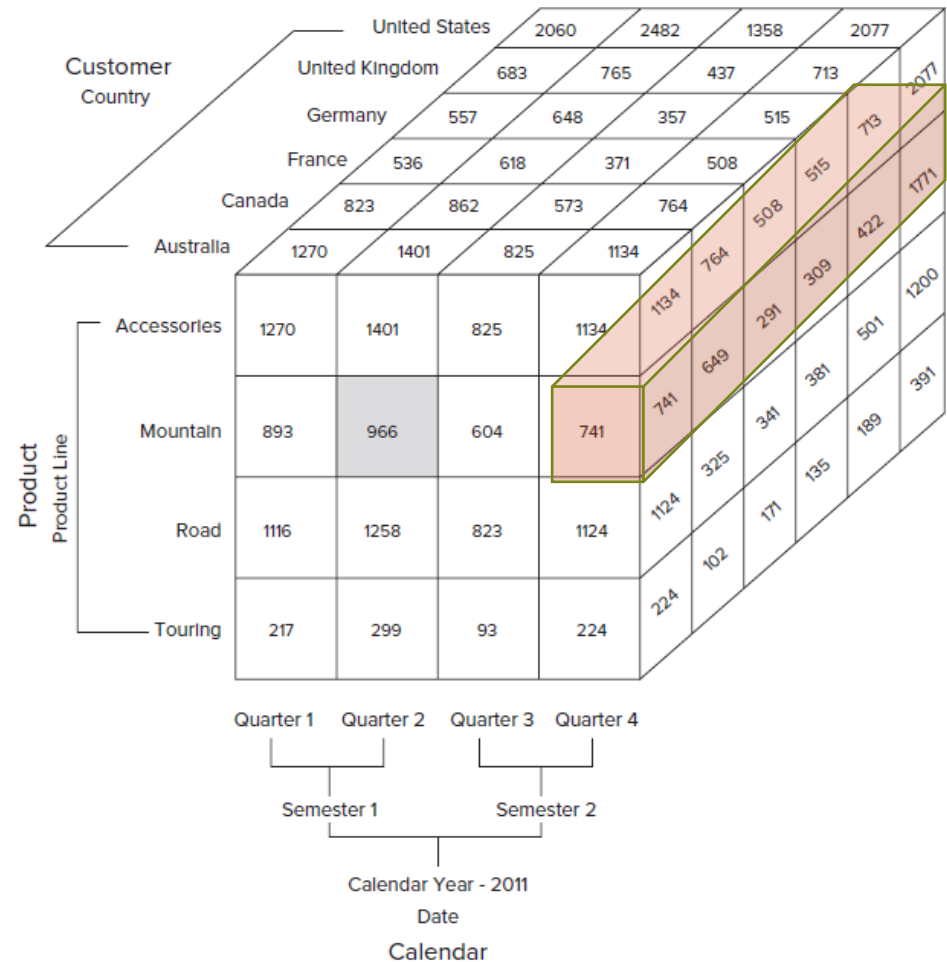


MDX

MDX CONCEPTS

Tuple

(
Product.Mountain,
Calendar.Quarter4
)

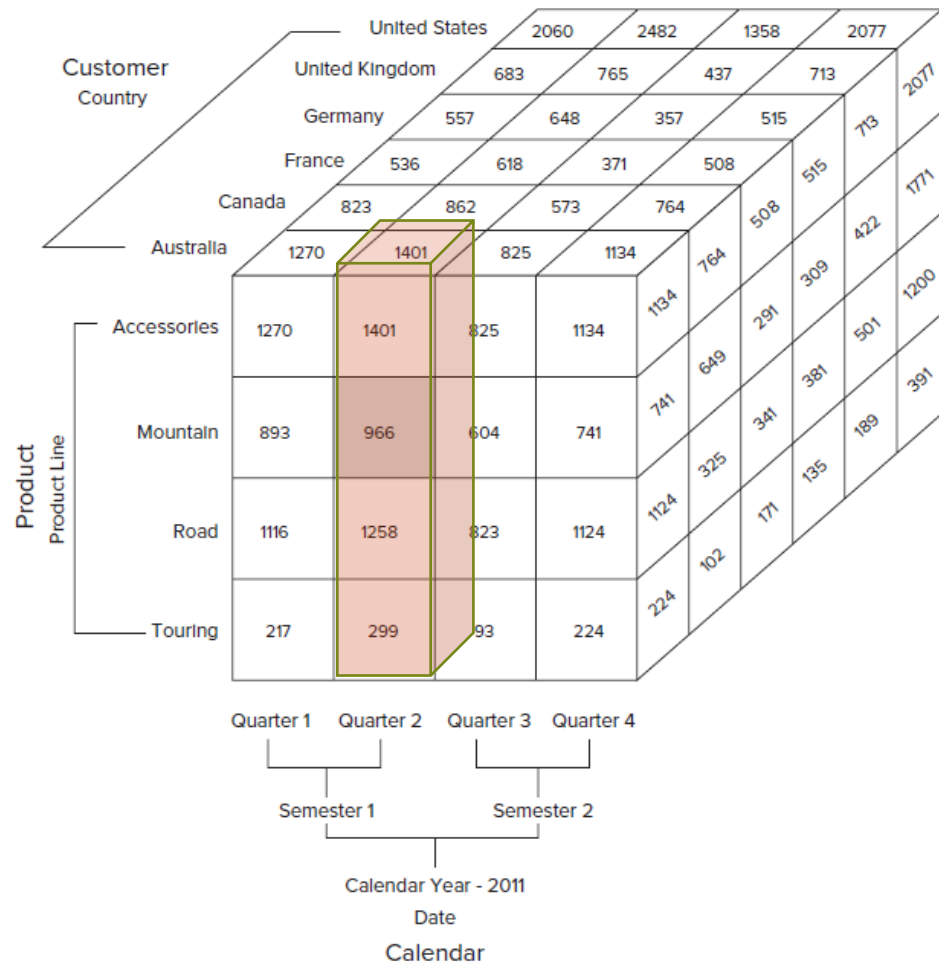


MDX

MDX CONCEPTS

Tuple

(
 Customer.Australia,
 Calendar.Quarter2
)



Step 1:
Resolve columns axis

Step 2:
Resolve rows axis

```
SELECT
{ ([Measures].[Sales Amount Without Tax], [Invoice Date].[Calendar Year].[CY2016]),
([Measures].[Sales Amount Without Tax], [Invoice Date].[Calendar Year].[CY2015]),
([Measures].[Sales Amount Without Tax], [Invoice Date].[Calendar Year].[CY2014]) }
ON COLUMNS,
{ ([Stock Item].[Color].[Black], [City].[Sales Territory].[Southwest]),
([Stock Item].[Color].[Blue], [City].[Sales Territory].[Far West]) }
ON ROWS
FROM
[Wide World Importers DW]
WHERE
([Customer].[Buying Group].[Tailspin Toys],
[Stock Item].[Size].[L])
);
```

100 %

		Sales Amount Without Tax	Sales Amount Without Tax	Sales Amount Without Tax
		CY2016	CY2015	CY2014
Black	Southwest	\$256.00	\$16,568.00	\$19,272.00
Blue	Far West	\$690.00	\$3,570.00	\$1,860.00

Step 3:
Construct & resolve
tuples for each cell

```
(([Measures].[Sales Amount Without Tax], [Invoice Date].[Calendar Year].[CY2016],
[Stock Item].[Color].[Blue], [City].[Sales Territory].[Far West])
```

MDX

Subcubes

The following MDX expression uses a subselect statement

```
SELECT
    [Date].[Calendar Year].MEMBERS on 0,
    [Product].[Product].MEMBERS on 1
FROM
    (SELECT
        TOPCOUNT(
            [Product].[Product].CHILDREN, 10,
            [Measures].[Sales Amount]) ON 0
        FROM [Adventure Works])
WHERE [Measures].[Sales Amount]
```

Defines a subcube with only top 10 products (based on Sales Amount measure)

This ,new' cube has an ALL member adjusted to these top 10 products

Without subcube

	All Periods	CY 2005	CY 2006	CY 2007	CY 2008
All Products	\$80,450,596.98	\$8,065,435.31	\$24,144,429.65	\$32,202,669.43	\$16,038,1

With subcube

	All Periods	CY 2005	CY 2006	CY 2007	CY 2008
All Products	\$19,997,183.30	\$1,696,815.63	\$2,816,611.28	\$7,930,797.72	\$7,552,958.

MDX

WITH Clause

Defining Named Sets and Calculated Members

WITH clause provides you with the ability to create calculations that must be formulated within the scope of a specific query

You can specify several calculations in one WITH clause

Comma separated

The syntax of the WITH clause is:

[WITH <formula_expression> [, <formula_expression> ...]]

MDX

WITH clause – Calculated Members

Calculated members allow one to define formulas and treat the formula as a new member of a specified parent

Parent refers to the parent of the new calculated member name

dimensions are organized as a hierarchy – position of calculated members must be defined inside the hierarchy

Syntax for a calculated member:

in front of the SELECT statement

WITH MEMBER parent.name AS 'expression,'

MDX

WITH Clause – Calculated Members

Calculated Members

Calculated members are calculations specified by MDX expressions

resolved as a result of MDX expression evaluation rather than just by the retrieval of the original fact data

The MDX_Expression should return a value that calculates the value of the member

The formula_expression of the WITH clause for calculated members is:

```
Formula_expression := MEMBER <MemberName> AS  
[']<MDX_Expression>[']
```

MDX

WITH Clause – Calculated Members

An example of a calculated member statement:

```
WITH MEMBER Measures.[Profit] AS
```

```
    [Measures].[Internet Sales Amount]-[Measures].[Internet Standard Product  
    Cost]
```

```
SELECT
```

```
    Measures.profit ON COLUMNS,
```

```
    [Customer].[Country].MEMBERS ON ROWS
```

```
FROM [Adventure Works]
```

MDX

WITH Clause – Calculated Members

Another MDX example

WITH MEMBER Measures.Test AS

([Measures].[Amount][Measures].[Count])*

SELECT

Time.Calendar on 0,

Measures.Test on 1

FROM

Sales

MDX

WITH Clause – Named Sets

Named Sets

a set expression, even though simple, can often be quite lengthy and might increase query complexity

to mitigate this issue you can define sets dynamically
assigning a name that is then used within the query

an alias for the collection of tuples in the set that can be used anywhere within the query

The formula_expression for the WITH clause with a named set is:

`formula_expression := SET <set_alias_name> AS ['<set>[,]`

Syntax for a named set:

In front of the SELECT statement

WITH SET set_name AS 'expression'

MDX

WITH Clause – Named Sets

Example:

```
WITH SET SetProds AS
```

```
    {[Product].[Prod].[Licence].[Personal],  
    [Product].[Prod].[Licence].[SuperCorporate]}
```

```
SELECT
```

```
    [Measures].[Amount] on columns,  
    SetProds on rows
```

```
FROM [Sales]
```

MDX

The simplest use of calculated members

Typically:

Defining a new measure that relates to already defined measures

Very often used to define non-additive measures that can be decomposed into additive counterparts

calculate the percentage profit for sales, by defining the calculated measure Profit Percent.

WITH MEMBER Measures.ProfitPercent AS

*'(Measures.[Store Sales] - Measures.[Store Cost]) / (Measures.[Store Cost])',
FORMAT_STRING = '#.00%'*

MDX

MDX Function Categories - Member Functions Hierarchical Navigation

.Functions

PREVMEMBER, NEXTMEMBER, CURRENTMEMBER

PARENT

FIRSTCHILD, LASTCHILD

CHILDREN (only for members)

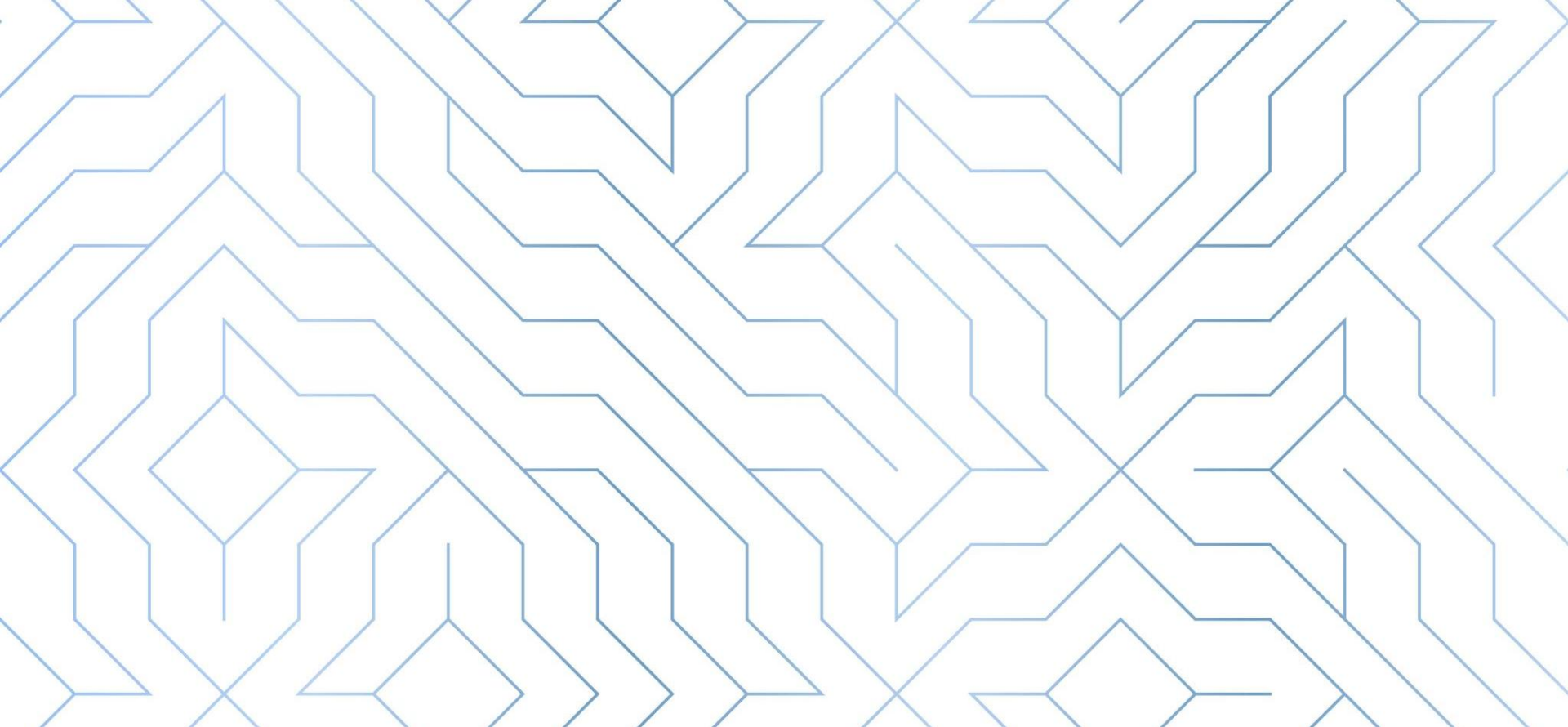
MEMBERS / ALLMEMBERS (only for levels)

MDX Functions

Order

Ranking

Set functions



Examples

MDX

```
SELECT  
{[CARS].[All CARS].[Chevy], [CARS].[All CARS].[Ford]} ON ROWS,  
{[DATE].[All DATE].[March], [DATE].[All DATE].[April]} ON COLUMNS  
FROM MDDBCARS;
```

	March	April
Chevy	\$155 000.00	\$ 75 000.00
Ford	\$ 55 000.00	\$175 000.00

MDX

```
SELECT  
{ [CARS].[ALL CARS].[CHEVY], [CARS].[ALL CARS].[FORD] } ON ROWS,  
{ [DATE].[ALL DATE].[MARCH], [DATE].[ALL DATE].[APRIL] } ON COLUMNS  
FROM MDDBCARS  
WHERE ( [MEASURES].[SALES_N] )
```

Sales_N	March	April
Chevy	1 000	700
Ford	600	1 500

MDX

```
SELECT  
{[CARS].[ALL CARS].[CHEVY], [CARS].[ALL CARS].[FORD]} ON COLUMNS,  
{[DATE].[ALL DATE].[JANUARY]:[DATE].[ALL DATE].[APRIL]} ON ROWS  
FROM MDDBCARS
```

	Chevy	Ford
January	\$66 000.00	\$ 79 000.00
February	\$55 000.00	\$ 72 000.00
March	\$155 000.00	\$ 55 000.00
April	\$ 75 000.00	\$175 000.00

MDX

```
SELECT  
{[CARS].[ALL CARS].[CHEVY], [CARS].[ALL CARS].[FORD]} ON COLUMNS,  
{[DATE].MEMBERS} ON ROWS  
FROM MDDBCARS
```

	Chevy	Ford
1998	\$566 000.00	\$ 479 000.00
1999	\$545 000.00	\$ 672 000.00
2000	\$745 000.00	\$ 527 000.00
2001	\$345 000.00	\$ 622 000.00

MDX

```
SELECT  
  {[CARS].[ALL CARS].[FORD].CHILDREN} ON COLUMNS,  
  {[DATE].MEMBERS} ON ROWS  
FROM MDDBCARS
```

	Ford Mustang	Ford Taurus	...
1998	\$56 000.00	\$ 79 000.00	
1999	\$54 000.00	\$ 72 000.00	
2000	\$72 000.00	\$ 52 000.00	
2001	\$34 000.00	\$ 22 000.00	

MDX

```
SELECT {([CARS].[ALL CARS].[CHEVY], [MEASURES].[SALES_SUM]),  
([CARS].[ALL CARS].[CHEVY], [MEASURES].[SALES_N]),  
([CARS].[ALL CARS].[FORD], [MEASURES].[SALES_SUM]),  
([CARS].[ALL CARS].[FORD], [MEASURES].[SALES_N])  
} ON COLUMNS,  
{[DATE].MEMBERS} ON ROWS  
FROM MDDBCARS
```

	Chevy		Ford	
	Sales_Sum	Sales_N	Sales_Sum	Sales_N
1998	\$566 000.00	450	\$ 479 000.00	450
1999	\$545 000.00	475	\$ 672 000.00	670
2000	\$745 000.00	750	\$ 527 000.00	490
2001	\$345 000.00	325	\$ 622 000.00	640

MDX

```
SELECT
{CROSSJOIN({[CARS].[ALL CARS].[CHEVY], [CARS].[ALL CARS].[FORD]},
{[MEASURES].[SALES_SUM], [MEASURES].[SALES_N]})
} ON COLUMNS,
{[DATE].MEMBERS} ON ROWS
FROM MDDBCARS
```

	Chevy		Ford	
	Sales_Sum	Sales_N	Sales_Sum	Sales_N
1998	\$566 000.00	450	\$ 479 000.00	450
1999	\$545 000.00	475	\$ 672 000.00	670
2000	\$745 000.00	750	\$ 527 000.00	490
2001	\$345 000.00	325	\$ 622 000.00	640

MDX

```
SELECT NON EMPTY [Store Type].[Store Type].MEMBERS ON COLUMNS,  
FILTER([Store].[Store City].MEMBERS, (Measures.[Profit], [Time].[1997])  
> 250000) ON ROWS FROM [Sales] WHERE (Measures.[Profit],  
[Time].[Year].[1997])
```

Profit	Normal	24 Hours
Toronto	\$66 000.00	\$196 000.00
Vancouver	\$111 000.00	\$156 000.00
New York	\$59 000.00	\$196 000.00
Chicago	\$75 000.00	\$211 000.00

MDX

```
SELECT Measures.MEMBERS ON COLUMNS,  
ORDER({ [Store].[Store City].MEMBERS}, Measures.[Sales Count], DESC) ON  
ROWS  
FROM [Sales]
```

	Profit	Sales Count	...
New York	\$747 000.00	2 196 000	
Chicago	\$785 000.00	1 956 000	
Toronto	\$666 000.00	1 916 000	
Vancouver	\$711 000.00	1 596 000	

MDX

```
WITH  
MEMBER [Time].[Year Difference] AS [Time].[2nd half] - [Time].[1st half]  
SELECT { [Account].[Income], [Account].[Expenses] } ON COLUMNS,  
{ [Time].[1st half], [Time].[2nd half], [Time].[Year Difference] } ON  
ROWS  
FROM [Financials]
```

	Income	Expenses
1st Half	5 000	4 200
2st Half	8 000	7 000
Year Difference	3 000	2 800

MDX

```
WITH  
MEMBER [Account].[Net Income] AS  
  ([Account].[Income] - [Account].[Expenses]) / [Account].[Income]  
SELECT { [Account].[Income], [Account].[Expenses], [Account].[Net  
Income] } ON COLUMNS,  
{ [Time].[1st half], [Time].[2nd half] } ON ROWS  
FROM [Financials]
```

	Income	Expenses	Net Income
1st Half	5 000	4 200	0.16
2st Half	8 000	7 000	0.125

Jensen C.S., Pedersen T.B., Thomsen C.,
Multidimensional Databases and Data Warehousing,
Morgan & Claypool Publishers series SYNTHESIS LECTURES ON DATA MANAGEMENT,
2010

Inmon W.,
Building the Data Warehouse,
John Wiley & Sons, New York 2002

Sherman R.,
Business Intelligence Guidebook
Elsevier, 2015

Claudia Imhoff, Nicholas Gallempo, Jonathan G. Geiger,
Mastering Data Warehouse Design - Relational and Dimensional Techniques,
Wiley Publishing, Inc., 2003

Efraim Turban, Ramesh Sharda, Dursun Delen, David King, Janine E. Aronson
Business Intelligence: A Managerial Approach
Prentice Hall, 2010

Sanjiv Purba (ed.)
Handbook of Data Management (1999 Edition)
Auerbach, 1999

Joy Arulraj
Lecture for Database System Implementation, 2024
Georgia Tech, College of Computing
<https://faculty.cc.gatech.edu/~jarulraj/courses/4420-f24/slides/22-columnar-storage-and-compression.pdf>

Bibliography

SOURCES